

## SUBARRAY SUM

```
#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>
using namespace std;

// Finds the maximum sum of a subarray
// crossing the middle.
long long maximumCrossingSum(const
vector<int>& nums, int left, int middle, int
right)
{
    long long sum = 0;
    long long maximumLeftSum =
LLONG_MIN;

    // Start from the middle and move left.
    for (int i = middle; i >= left; i--) {
        sum += nums[i];
        maximumLeftSum =
max(maximumLeftSum, sum);
    }

    sum = 0;
    long long maximumRightSum =
LLONG_MIN;

    // Start after the middle and move right.
    for (int i = middle + 1; i <= right; i++) {
        sum += nums[i];
        maximumRightSum =
max(maximumRightSum, sum);
    }

    return maximumLeftSum +
maximumRightSum;
}

long long maximumSubarraySum(const
vector<int>& nums, int left, int right)
{
    // Base case: only one element.
    if (left == right)
    {
        return nums[left];
    }

    int middle = left + (right - left) / 2;

    long long leftMaximum =
maximumSubarraySum(nums, left,
middle);

    long long rightMaximum =
maximumSubarraySum(nums, middle
+ 1, right);

    long long crossingMaximum =
maximumCrossingSum(nums, left,
middle, right);

    return max({leftMaximum,
rightMaximum, crossingMaximum});
}

int main() {
    vector<int> nums = {
        -2, 1, -3, 4, -1, 2, 1, -5, 4
    };

    if (nums.empty()) {
        cout << "The array is empty.\n";
        return 0;
    }

    long long answer =
maximumSubarraySum(nums, 0,
nums.size() - 1);

    cout << "Maximum subarray sum: " <<
answer << "\n";

    return 0;
}
```

## NICE SUBSTRING

```
#include <iostream>
#include <unordered_set>
#include <using namespace std>;
string longestNiceSubstring(const string& text) { // A
nice substring must contain at least two characters. if
(text.size() < 2) { return ""; }
unordered_set<char> characters;

    for (char ch : text) {
        characters.insert(ch);
    }

    for (int i = 0; i < static_cast<int>(text.size()); i++)
    {
        char current = text[i];

        char oppositeCase;

        if (islower(static_cast<unsigned char>(current)))
        {
            oppositeCase =
static_cast<char>(toupper(static_cast<unsigned
char>(current)));
        }
        else
        {
            oppositeCase =
static_cast<char>(tolower(static_cast<unsigned
char>(current)));
        }

        // The current character does not have
        // its opposite case in this substring.
        if (!characters.count(oppositeCase))
        {
            string leftAnswer =
longestNiceSubstring(text.substr(0, i));

            string rightAnswer =
longestNiceSubstring(text.substr(i + 1));

            // Return the left one when both lengths are equal.
            if (leftAnswer.size() >= rightAnswer.size())
            {
                return leftAnswer;
            }
            return rightAnswer;
        }
    }

    // Every character has both uppercase and lowercase
    forms.
    return text;
}

int main() { string text = "YazaAay";
cout << "Longest nice substring: "
<< longestNiceSubstring(text)
<< "\n";

return 0;
}
```

## CONVEX HULL

```
#include <iostream>
#include <vector>
#include <set>
#include <cstdlib>
using namespace std;

struct Point {
    long long x;
    long long y;

    bool operator<(const Point& other) const {
        if (x != other.x) {
            return x < other.x;
        }
        return y < other.y;
    };
};

// Cross product of vectors AB and AP.
long long crossProduct(
const Point& a,
const Point& b,
const Point& p
){
    return (b.x - a.x) * (p.y - a.y)
        - (b.y - a.y) * (p.x - a.x);
}
```

```
// Returns the side of point P relative to line AB.
int findSide(
const Point& a,
const Point& b,
const Point& p
){
    long long value = crossProduct(a, b, p);

    if (value > 0) {
        return 1;
    }

    if (value < 0) {
        return -1;
    }

    return 0;
}

// Returns a value proportional to the distance
// between point P and line AB.
long long lineDistance(
const Point& a,
const Point& b,
const Point& p
){
    return llabs(crossProduct(a, b, p));
}

void buildHull(
const vector<Point>& points,
const Point& a,
const Point& b,
int side,
set<Point>& hull
){
    int farthestIndex = -1;
    long long maximumDistance = 0;

    for (int i = 0; i < static_cast<int>(points.size()); i++) {
        if (
            findSide(a, b, points[i]) == side &&
            distance > maximumDistance
        ) {
            farthestIndex = i;
            maximumDistance = distance;
        }
    }

    // No point exists on the required side.
    // A and B are hull points.
    if (farthestIndex == -1) {
        hull.insert(a);
        hull.insert(b);
        return;
    }

    Point farthestPoint = points[farthestIndex];

    // Recursively process the two new regions.
    buildHull(
        points,
        farthestPoint,
        a,
        -findSide(farthestPoint, a, b),
        hull
    );

    buildHull(
        points,
        farthestPoint,
        b,
        -findSide(farthestPoint, b, a),
        hull
    );
}

set<Point> quickHull(const vector<Point>& points) {
    set<Point> hull;

    if (points.size() < 3) {
        for (const Point& point : points) {
            hull.insert(point);
        }

        return hull;
    }

    int minimumXIndex = 0;
    int maximumXIndex = 0;

    for (int i = 1; i < static_cast<int>(points.size()); i++) {
        if (points[i].x < points[minimumXIndex].x) {
            minimumXIndex = i;
        }
    }
}
```

```
if (points[i].x >
points[maximumXIndex].x) {
    maximumXIndex = i;
}

    Point leftmost =
points[minimumXIndex];
    Point rightmost =
points[maximumXIndex];

    // Find hull points above line leftmost-
rightmost.
    buildHull(
        points,
        leftmost,
        rightmost,
        1,
        hull
    );

    // Find hull points below line leftmost-
rightmost.
    buildHull(
        points,
        leftmost,
        rightmost,
        -1,
        hull
    );

    return hull;
}

int main() {
    vector<Point> points = {
        {0, 3},
        {2, 2},
        {1, 1},
        {2, 1},
        {3, 0},
        {0, 0},
        {3, 3}
    };

    set<Point> hull = quickHull(points);

    cout << "Points on the convex hull:\n";

    for (const Point& point : hull) {
        cout << "(" << point.x
            << ", " << point.y << ")" << "\n";
    }

    return 0;
}
```

